

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 14
“SORTING”



DISUSUN OLEH:
AMMAR ATHAZZAM AL-AYYUBI
109082500012
S1 IF-13-02
DOSEN:
WAHYU ANDI SAPUTRA, S.PD., M.ENG.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Dasar teori sorting atau pengurutan data pada data terstruktur dalam materi ini berfokus pada dua pendekatan algoritma klasik, yaitu Selection Sort dan Insertion Sort. Selection Sort bekerja dengan cara memindai seluruh sisa data untuk mencari indeks dengan nilai ekstrim baik itu nilai terkecil untuk urutan membesar (ascending) maupun nilai terbesar lalu menukarkan posisi nilai ekstrim tersebut dengan elemen yang berada di ujung rentang data yang belum terurut.

Sementara itu, Insertion Sort bekerja dengan analogi menyisipkan kartu, di mana algoritma ini mengambil satu elemen yang belum terurut lalu mencarinya yang tepat secara sekuensial pada bagian data yang sudah terurut sebelumnya sambil menggeser elemen-elemen lain ke kanan untuk membuka ruang kosong.

Kedua konsep pengurutan ini tidak hanya dapat diterapkan pada array dengan tipe data dasar seperti bilangan bulat, melainkan juga sangat adaptif untuk mengurutkan tipe data bentukan (struct) dengan cara membandingkan field atau atribut spesifik yang ada di dalamnya, seperti mengurutkan data mahasiswa berdasarkan IPK atau data buku berdasarkan ratingnya.

SOAL LATIHAN

Insertion Sort

1.

Source Code:

```
package main
import "fmt"
func insertionSortAsc(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
        temp := arr[i]
        j := i
        for j > 0 && temp < arr[j-1] {
            arr[j] = arr[j-1]
            j--
        }
        arr[j] = temp
    }
}
func main() {
    var val int
    var data []int
    for {
        fmt.Scan(&val)
        if val < 0 {
            break
        }
        data = append(data, val)
    }
    if len(data) == 0 {
        fmt.Println("Tidak ada data untuk diproses.")
        return
    }
    insertionSortAsc(data)
    for i, v := range data {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(v)
    }
    fmt.Println()
    if len(data) < 2 {
        fmt.Println("Data berjarak 0")
        return
    }
    jarakSama := true
    selisih := data[1] - data[0]
    for i := 1; i < len(data)-1; i++ {
        if data[i+1]-data[i] != selisih {
```

```

        jarakSama = false
        break
    }
}
if jarakSama {
    fmt.Printf("Data berjarak %d\n", selisih)
} else {
    fmt.Println("Data berjarak tidak tetap")
}
}

```

Output:

```

PS D:\golang\modul14> go run 1.go
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS D:\golang\modul14> go run 1.go
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS D:\golang\modul14> 

```

Deskripsi Program:

Program ini digunakan untuk membaca sekumpulan bilangan bulat positif dari pengguna yang diakhiri dengan bilangan negatif, lalu mengurutkannya dari yang terkecil ke terbesar menggunakan metode insertion sort, serta memeriksa apakah selisih antar angka yang telah terurut tersebut selalu konstan atau berubah-ubah.

Input dimasukkan berupa deretan angka acak dipisahkan spasi dan ditutup dengan satu angka negatif. Program akan mengeksekusi input tersebut dengan cara membuang angka negatif, menyisipkan setiap angka non-negatif ke posisi yang benar hingga terurut menaik, lalu membandingkan setiap selisih antar dua angka yang berdekatan untuk menentukan dan mencetak status jaraknya.

2.

Source Code:

```
package main
import "fmt"
const nMax = 7919
type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}
type DaftarBuku struct {
    pustaka [nMax]Buku
    nPustaka int
}
func DaftarkanBuku(pustaka *DaftarBuku, n int) {
    pustaka.nPustaka = n
    for i := 0; i < n; i++ {
        fmt.Scan(&pustaka.pustaka[i].id,
                &pustaka.pustaka[i].judul,
                &pustaka.pustaka[i].penulis,
                &pustaka.pustaka[i].penerbit,
                &pustaka.pustaka[i].eksemplar,
                &pustaka.pustaka[i].tahun,
                &pustaka.pustaka[i].rating)
    }
}
func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        fmt.Println("Tidak ada buku.")
        return
    }
    idxMax := 0
    for i := 1; i < n; i++ {
        if pustaka.pustaka[i].rating >
pustaka.pustaka[idxMax].rating {
            idxMax = i
        }
    }
    fav := pustaka.pustaka[idxMax]
    fmt.Printf("Terfavorit: %s, %s, %s, %d\n", fav.judul,
fav.penulis, fav.penerbit, fav.tahun)
}
func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        temp := pustaka.pustaka[i]
        j := i
        for j > 0 && temp.rating > pustaka.pustaka[j-
1].rating {
            pustaka.pustaka[j] = pustaka.pustaka[j-1]
            j--
        }
    }
}
```

```

        pustaka.pustaka[j] = temp
    }
}
func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5
    if n < 5 {
        limit = n
    }
    fmt.Println("5 Buku Rating Tertinggi:")
    for i := 0; i < limit; i++ {
        fmt.Printf("%d. %s (Rating: %d)\n", i+1,
pustaka.pustaka[i].judul, pustaka.pustaka[i].rating)
    }
}
func CariBuku(pustaka DaftarBuku, n int, r int) {
    low := 0
    high := n - 1
    foundIdx := -1
    for low <= high {
        mid := (low + high) / 2
        if pustaka.pustaka[mid].rating == r {
            foundIdx = mid
            break
        } else if pustaka.pustaka[mid].rating < r {
            high = mid - 1
        } else {
            low = mid + 1
        }
    }
    if foundIdx != -1 {
        b := pustaka.pustaka[foundIdx]
        fmt.Printf("Buku Ditemukan: %s, %s, %s, %d, %d,
%d\n",
            b.judul, b.penulis, b.penerbit, b.tahun,
b.eksemplar, b.rating)
    } else {
        fmt.Println("Tidak ada buku dengan rating seperti
itu")
    }
}
func main() {
    var bPustaka DaftarBuku
    var n, targetRating int
    fmt.Scan(&n)
    DaftarkanBuku(&bPustaka, n)
    CetakTerfavorit(bPustaka, n)
    UrutBuku(&bPustaka, n)
    Cetak5Terbaru(bPustaka, n)
    fmt.Scan(&targetRating)
    CariBuku(bPustaka, n, targetRating)
}

```

Output:

```
PS D:\golang\modul14> go run 2.go
5
B01 StrukturData Ali AndiPublisher 10 2023 90
B02 Alpro2 Bob BinusPress 5 2024 85
B03 Kalkulus Charlie Erlangga 7 2022 95
B04 Struktur_Data Dian_Pratiwi Andipublisher 12 2023 88
B05 Jaringan_Komputer Edo_Wijaya GNS3_Media 8 2025 92
Terfavorit: Kalkulus, Charlie, Erlangga, 2022
5 Buku Rating Tertinggi:
1. Kalkulus (Rating: 95)
2. Jaringan_Komputer (Rating: 92)
3. StrukturData (Rating: 90)
4. Struktur_Data (Rating: 88)
5. Alpro2 (Rating: 85)
92
Buku Ditemukan: Jaringan_Komputer, Edo_Wijaya, GNS3_Media, 2025, 8, 92
PS D:\golang\modul14> 
```

Deskripsi Program:

Program ini digunakan untuk mengelola dan merapikan data buku di perpustakaan berdasarkan penilaian atau ratingnya menggunakan kombinasi metode pengurutan insertion sort dan pencarian biner.

Input dimasukkan berupa total jumlah buku, diikuti detail data setiap buku seperti ID, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating, lalu diakhiri dengan angka rating yang ingin dicari. Program mengeksekusi input ini dengan cara membaca dan menyimpan seluruh data buku ke dalam memori, mencari dan menampilkan buku terfavorit dengan rating tertinggi, mengurutkan seluruh daftar buku secara menurun berdasarkan ratingnya, menampilkan maksimal lima judul buku berperingkat teratas, serta melakukan pencarian cepat untuk memunculkan data buku yang memiliki rating sesuai dengan angka pencarian di baris terakhir.

DAFTAR PUSTAKA